

Competitive Programming Notebook

py.Sandu

Contents

1	Graph	2
1.1	Dial	2
1.2	Bellmanford	2
1.3	Kruskal	2
1.4	Tarjan	2
1.5	Floydwarshall	3
1.6	Eulereanpath	3
1.7	Dijkstra	3
1.8	Kahn	4
1.9	Lca	4
2	Geometry	4
2.1	Convexhull	4
2.2	Point	4
2.3	Pointinpolygon	5
3	.template	6
3.1	Template	6
4	String	6
4.1	Hash	6
5	DP	6
5.1	Lis	6
5.2	Countingdp	6
5.3	Knapsack	6
6	Math	7
6.1	Extendedeuclidean	7
6.2	Crivo	7
6.3	Fastexponentiation	7
6.4	Matrixexponentiation	7
6.5	Pie	8
6.6	Factorial	8
7	DS	8
7.1	Bit2d	8
7.2	Segtree	8
7.3	Seglazy	9
7.4	Psum2d	9
7.5	Dsu	9
7.6	Sparsetable	10
7.7	Bit	10
7.8	Monotonicstack	10

1 Graph

1.1 Dial

```

1 // Dial, optimized dijkstra for low W
2 // Time-complexity:  $O(E + V \cdot W)$ ,  $W$  : max weight
3
4 vector<int> dial(int n, int w, int src, vector<vector<pii>> &adj){
5
6     vector<int> dist(n+1, INF);
7     vector<queue<int>> a(w+1);
8     dist[src] = 0; a[0].push(src);
9     int pos = 0, qnt = 1;
10
11     while(qnt){
12         while(a[pos%(w+1)].empty()) pos++;
13
14         int i = a[pos%(w+1)].front(); a[pos%(w+1)].pop(); qnt--;
15
16         if(dist[i] < pos) continue;
17
18         for(auto [u, d_u] : adj[i]){
19             if(dist[i] + d_u < dist[u]){
20                 dist[u] = dist[i] + d_u;
21                 a[(dist[u])%(w+1)].push(u); qnt++;
22             }
23         }
24     }
25
26     return dist;
27 }
28
29 // especial case: max W = 1
30 vector<int> bfs01(int n, int src, vector<vector<pii>> &adj){
31
32     vector<int> dist(n+1, INF);
33     deque<int> dq;
34
35     dist[src] = 0;
36     dq.push_front(src);
37
38     while(!dq.empty()){
39         int i = dq.front();
40         dq.pop_front();
41
42         for(auto [u, w] : adj[i]){
43             if(dist[i] + w < dist[u]){
44                 dist[u] = dist[i] + w;
45                 if(w == 0) dq.push_front(u);
46                 else dq.push_back(u);
47             }
48         }
49     }
50
51     return dist;
52 }
```

1.2 Bellmanford

```

1 // Find the shortest path from a source vertex to all
   other vertices in a graph with negative weight
   edges and detect negative weight cycles.
2 // Time Complexity:  $O(V \cdot E)$ 
3 // Space Complexity:  $O(V)$ 
4 // Use cases: SSSP with negative edges, negative
   cycle detection, system of difference constraints
   , shortest path with at most K edges
5
```

```

6 bool bellmanFord(vector<vector<pair<int, int>>> &adj,
   int src) {
7
8     int n = adj.size() - 1;
9     vector<int> dist(n+1, INF);
10    dist[src] = 0;
11
12    // Relax all edges n-1 times
13    for (int i = 1; i < n; i++) {
14        for (int u = 1; u <= n; u++) {
15            for (auto& [v, w] : adj[u]) {
16                if (dist[u] != INF and dist[u] + w <
17                    dist[v]) {
18                    dist[v] = dist[u] + w;
19                }
20            }
21        }
22    }
23
24    // Check for negative weight cycles
25    for (int u = 1; u <= n; u++) {
26        for (auto& [v, w] : adj[u]) {
27            if (dist[u] != INF and dist[u] + w < dist
28                [v]) {
29                // Graph contains negative weight
30                cycle
31                return true;
32            }
33        }
34    }
35
36    return false;
37 }
```

1.3 Kruskal

```

1 // Kruskal algorithm for finding the Minimum Spanning
   Tree
2 // Time Complexity:  $O(E \log E)$ 
3 // Space complexity:  $O(V + E)$ 
4 // Use cases: connect nodes with min/max cost,
   clustering, minimax path
5
6 // edges: {weight, u, v}
7 vector<vector<pair<int, int>>> kruskalMST(vector<
   array<int, 3>>& edges, int n){
8     int m = (int)edges.size();
9     sort(edges.begin(), edges.end());
10    vector<vector<pair<int, int>>> mst(n+1);
11    DSU dsu(n+1); int cost = 0;
12    for(int i=0; i<m; i++){
13        int w = edges[i][0], a = edges[i][1], b =
14            edges[i][2];
15        if(dsu.merge(a, b)){
16            mst[a].push_back({w, b});
17            mst[b].push_back({w, a});
18            cost += w;
19        }
20    }
21    return mst;
22 }
```

1.4 Tarjan

```

1 // Finds Strongly Connected Components (SCCs) in a
   directed graph.
2 // Time Complexity:  $O(V + E)$ 
3 // Space Complexity:  $O(V)$ 
4 // Use cases: 2-SAT, shrinking cycles in directed
   graphs to form a DAG, finding reachability
   components.
5 // Note: SCC IDs are assigned in reverse topological
   order of the condensed DAG.
```

```

6
7 vector<int> tarjanSCC(const vector<vector<int>>& adj)
8 {
9     int n = adj.size() - 1;
10
11     vector<int> tin(n + 1, -1), low(n + 1, -1),
12     scc_id(n + 1, -1);
13     vector<bool> in_stack(n + 1, false);
14     stack<int> st;
15     int timer = 0, scc_cnt = 0;
16
17     auto dfs = [&](auto& self, int u) -> void {
18         tin[u] = low[u] = ++timer;
19         st.push(u);
20         in_stack[u] = true;
21
22         for (int v : adj[u]) {
23             if (tin[v] == -1) {
24                 self(self, v);
25                 low[u] = min(low[u], low[v]);
26             } else if (in_stack[v]) {
27                 low[u] = min(low[u], tin[v]);
28             }
29         }
30
31         if (low[u] == tin[u]) {
32             while (true) {
33                 int v = st.top();
34                 st.pop();
35                 in_stack[v] = false;
36                 scc_id[v] = scc_cnt;
37                 if (u == v) break;
38             }
39             scc_cnt++;
40         }
41     };
42
43     for (int i = 1; i <= n; i++) {
44         if (tin[i] == -1) {
45             dfs(dfs, i);
46         }
47     }
48
49     // retorna o vetor contendo a qual SCC cada
50     // vertice pertence.
51     // o total de SCCs gerados Ã sempre: *
52     max_element(scc_id.begin(), scc_id.end()) + 1
53     return scc_id;
54 }

```

1.5 Floydwarshall

```

1 // Find the shortest paths in a weighted graph with
2 // positive or negative edge weights (but with no
3 // negative cycles).
4 // Time Complexity: O(V^3)
5 // Space Complexity: O(V^2)
6 // Use cases: reachability, minimax distance, global
7 // negative cycle detection(dist[i][i] < 0)
8
9 void floydWarshall(vector<vector<int>>& dist) { //
10     dist(INF)
11
12     int n = dist.size() - 1;
13     for(int i = 1; i <= n; i++) dist[i][i] = 0;
14
15     for (int k = 1; k <= n; k++) {
16         for (int i = 1; i <= n; i++) {
17             for (int j = 1; j <= n; j++) {
18                 if (dist[i][k] != INF and dist[k][j]
19                 != INF) {

```

```

15         dist[i][j] = min(dist[i][j], dist
16         [i][k] + dist[k][j]);
17     }
18 }
19 }
20 }

```

1.6 Eulereanpath

```

1 // Find the Eulerian path in a graph using Hierholzer
2 // 's algorithm
3 // Time complexity: O(E) where E is the number of
4 // edges in the graph
5 // Use cases: visit every edge once, DNA sequencing,
6 // puzzles, drawing figures
7
8 vector<int> findEulerianPath(vector<vector<int>>& adj
9 , int start) {
10
11     vector<int> path;
12     stack<int> s;
13     s.push(start);
14
15     while (!s.empty()) {
16         int i = s.top();
17         if (adj[i].empty()) {
18             path.push_back(i);
19             s.pop();
20         } else {
21             int u = adj[i].back(); adj[i].pop_back();
22             s.push(u);
23         }
24     }
25
26     reverse(path.begin(), path.end());
27     return path;
28 }

```

1.7 Dijkstra

```

1 // Find shortest path from a source vertex to all
2 // other vertices in a graph with non-negative edge
3 // weights
4 // Time Complexity: O((V + E) log V)
5 // Space Complexity: O(V)
6 // Use cases: shortest path in non-negative weighted
7 // graphs
8
9 vector<int> dijkstra(vector<vector<pair<int, int>>&
10 adj, int src) {
11
12     int n = adj.size() - 1;
13     vector<int> dist(n+1, INF);
14     dist[src] = 0;
15
16     priority_queue<pair<int, int>, vector<pair<int,
17     int>>, greater<pair<int, int>>> pq; // Min-heap
18     priority queue
19     pq.push({0, src});
20
21     while (!pq.empty()) {
22         auto [d, i] = pq.top();
23         pq.pop();
24
25         if(d > dist[i]) continue;
26
27         for (auto [u, w] : adj[i]) {
28             if (dist[i] + w < dist[u]) {
29                 dist[u] = dist[i] + w;

```

```

26         pq.push({dist[u], u});
27     }
28 }
29 }
30
31 return dist;
32 }

```

1.8 Kahn

```

1 // Topological Sort (Kahn's Algorithm)
2 // Time Complexity: O(V + E)
3 // Space Complexity: O(V)
4 // Use cases: Dependency resolution, cycle detection
  in directed graphs, DP on DAGs
5
6 vector<int> TopologicalSort(vector<vector<int>>& adj,
  int n){
7     vector<int> degree(n+1, 0);
8     for(int i=1; i<=n; i++){
9         for(auto u: adj[i]) degree[u]++;
10    }
11
12    // priority_queue in case of lexicographically
  smallest answer
13    queue<int> q; vector<int> ans;
14    for(int i=1; i<=n; i++){
15        if(degree[i] == 0){
16            q.push(i); ans.push_back(i);
17        }
18    }
19
20
21    while(!q.empty()){
22        int i = q.front(); q.pop();
23        for(auto u: adj[i]){
24            degree[u]--;
25            if(degree[u] == 0){
26                q.push(u); ans.push_back(u);
27            }
28        }
29    }
30
31    if((int)ans.size() != n) return {};
32    return ans;
33 }
34 }

```

1.9 Lca

```

1 // Finds the Lowest Common Ancestor (LCA) of two
  nodes in a tree using Binary Lifting and Euler
  Tour technique.
2 // Time Complexity: O(N log N) pre-processing, O(log
  N) per query
3 // Space Complexity: O(N log N)
4 // Use cases: LCA queries, distance between two nodes
  in a tree, path queries on a tree, jumping k-th
  ancestors.
5
6 int timer, k;
7 vector<vector<int>> adj;
8 vector<vector<int>> up;
9 vector<int> tin, tout;
10
11 void dfs(int v, int p){
12     tin[v] = ++timer;
13     up[v][0] = p;
14     for(int i=1; i <= k; i++){
15         up[v][i] = up[up[v][i-1]][i-1];
16     }
17     for(int u: adj[v]){

```

```

18         if(u != p) dfs(u, v);
19     }
20     tout[v] = ++timer;
21 }
22
23 bool is_ancestor(int u, int v){
24     return tin[u] <= tin[v] and tout[u] >= tout[v];
25 }
26
27 int lca(int u, int v){
28     if(is_ancestor(u, v)) return u;
29     if(is_ancestor(v, u)) return v;
30
31     for(int i=k; i >= 0; i--){
32         if(!is_ancestor(up[u][i], v)) u = up[u][i];
33     }
34     return up[u][0];
35 }
36
37 void pre_process(int n, int root){
38     tin.resize(n+1);
39     tout.resize(n+1);
40     timer = 0;
41     k = __lg(n+1);
42     up.assign(n+1, vector<int>(k+1));
43     dfs(root, root);
44 }

```

2 Geometry

2.1 Convexhull

```

1
2 // pontos colineares na borda, mude <= EPS para < -
  EPS.
3 vector<Point> convex_hull(vector<Point>& pts) {
4     int n = pts.size(), k = 0;
5     if (n <= 2) return pts;
6
7     vector<Point> h(2 * n);
8     sort(pts.begin(), pts.end());
9
10    // lower hull
11    for (int i = 0; i < n; ++i) {
12        while (k >= 2 && cross_product(h[k-2], h[k-1],
  pts[i]) <= EPS) k--;
13        h[k++] = pts[i];
14    }
15
16    // upper hull
17    for (int i = n - 2; i >= 0; i--) {
18        while (k >= t && cross_product(h[k-2], h[k-1],
  pts[i]) <= EPS) k--;
19        h[k++] = pts[i];
20    }
21
22    h.resize(k - 1); // remove o ponto duplicado no
  final
23    return h;
24 }

```

2.2 Point

```

1
2 const double EPS = 1e-9;
3 // int -> int EPS = 0
4 // int -> mudar todo double pra inteiro, exceto
  funcao area
5 struct Point {
6
7     double x, y;

```

```

8
9 Point() : x(0), y(0) {}
10 Point(double x, double y) : x(x), y(y) {}
11
12 Point operator+(const Point& p) const { return
13 Point(x + p.x, y + p.y); }
14 Point operator-(const Point& p) const { return
15 Point(x - p.x, y - p.y); }
16 Point operator*(double c) const { return Point(x
17 * c, y * c); }
18 Point operator/(double c) const { return Point(x
19 / c, y / c); }
20
21 bool operator<(const Point& p) const {
22 if (abs(x - p.x) > EPS) return x < p.x;
23 return y < p.y - EPS;
24 }
25
26 bool operator==(const Point& p) const {
27 return abs(x - p.x) <= EPS && abs(y - p.y) <=
28 EPS;
29 }
30
31 // > 0: B esq. vetor OA, < 0: B dir. vetor OA, 0:
32 // Colineares
33 double cross_product(Point O, Point A, Point B) {
34 return (A.x - O.x) * (B.y - O.y) - (A.y - O.y) *
35 (B.x - O.x);
36 }
37
38 // distancia euclidiana
39 double dist(Point A, Point B) {
40 return hypot(A.x - B.x, A.y - B.y);
41 }
42
43 // distancia quadrada
44 double dist2(Point A, Point B) {
45 return (A.x - B.x) * (A.x - B.x) + (A.y - B.y) *
46 (A.y - B.y);
47 }
48
49 // produto escalar, dot == 0 perpendiculares
50 double dot_product(Point O, Point A, Point B) {
51 return (A.x - O.x) * (B.x - O.x) + (A.y - O.y) *
52 (B.y - O.y);
53 }
54
55 int get_half(Point O, Point P) {
56 if (P.y - O.y > EPS or (abs(P.y - O.y) <= EPS and
57 P.x - O.x > EPS)) return 1;
58 return 0;
59 }
60
61 // ordenacao por perimetro
62 void sort_by_angle(vector<Point>& p, Point center) {
63 sort(p.begin(), p.end(), [&](const Point& A,
64 const Point& B) {
65 int halfA = get_half(center, A);
66 int halfB = get_half(center, B);
67
68 if (halfA != halfB) {
69 return halfA > halfB;
70 }
71
72 double cross = cross_product(center, A, B);
73 if (abs(cross) > EPS) {
74 return cross > 0;
75 }
76
77 return dist2(center, A) < dist2(center, B) -
78 EPS;
79 });
80
81 }
82
83 // shoelace formula, vertices do poligono ordenados
84 // pelo perimetro.
85 double polygon_area(const vector<Point>& p) {
86 double area = 0.0;
87 int n = p.size();
88
89 for (int i = 0; i < n; i++) {
90 int next_i = (i + 1) % n;
91 area += (p[i].x * p[next_i].y) - (p[next_i].x
92 * p[i].y);
93 }
94
95 // return abs(area) -> int
96 return abs(area) / 2.0;
97 }
98
99 // aux func
100 int orientation(Point O, Point A, Point B) {
101 double val = cross_product(O, A, B);
102 if (abs(val) <= EPS) return 0; // Colinear
103 return (val > 0) ? 1 : -1; // 1: Esquerda,
104 -1: Direita
105 }
106
107 // 2. P in seg AB?
108 bool on_segment(Point P, Point A, Point B) {
109 if (abs(cross_product(A, B, P)) > EPS) return
110 false;
111 return P.x >= min(A.x, B.x) - EPS && P.x <= max(A
112 .x, B.x) + EPS &&
113 P.y >= min(A.y, B.y) - EPS && P.y <= max(A
114 .y, B.y) + EPS;
115 }
116
117 // Ab seg cross CD?
118 bool segments_intersect(Point A, Point B, Point C,
119 Point D) {
120 // calcula as 4 orientaões necessárias
121 int o1 = orientation(A, B, C);
122 int o2 = orientation(A, B, D);
123 int o3 = orientation(C, D, A);
124 int o4 = orientation(C, D, B);
125
126 if (o1 != o2 && o3 != o4) {
127 return true;
128 }
129
130 if (o1 == 0 && on_segment(C, A, B)) return true;
131 // C toca AB
132 if (o2 == 0 && on_segment(D, A, B)) return true;
133 // D toca AB
134 if (o3 == 0 && on_segment(A, C, D)) return true;
135 // A toca CD
136 if (o4 == 0 && on_segment(B, C, D)) return true;
137 // B toca CD
138
139 return false; // nao se cruzam
140 }
141
142 // retorna: 1 (Dentro), -1 (Na borda), 0 (Fora)
143 // o poligono nao precisa ser convexo, mas os
144 // vertices devem estar em ordem.
145 int point_in_polygon(Point P, const vector<Point>&
146 poly) {
147 int n = poly.size();
148 bool in = false;
149
150 for (int i = 0; i < n; i++) {
151 Point A = poly[i];

```

2.3 Pointinpolygon

```

1 // retorna: 1 (Dentro), -1 (Na borda), 0 (Fora)
2 // o poligono nao precisa ser convexo, mas os
3 // vertices devem estar em ordem.
4 int point_in_polygon(Point P, const vector<Point>&
5 poly) {
6 int n = poly.size();
7 bool in = false;
8
9 for (int i = 0; i < n; i++) {
10 Point A = poly[i];

```

```

9      Point B = poly[(i + 1) % n];
10     if (on_segment(P, A, B)) return -1;
11     if (A.y > B.y) swap(A, B);
12     if (A.y <= P.y + EPS && P.y + EPS < B.y) {
13         if (cross_product(A, B, P) > EPS) {
14             in = !in;
15         }
16     }
17 }
18
19 return in ? 1 : 0;
20 }

```

3 .template

3.1 Template

```

1 #include<bits/stdc++.h>
2
3 using namespace std;
4
5 #define int long long
6 #define vi vector<int>
7 #define pii pair<int, int>
8 #define all(x) (x).begin(), (x).end()
9 #define debug(x...) cout<<"["#x"]: ",[](auto...$){((
10     cout<<$<<" "; ...);}(x),cout<<endl
11
12 void solve(){
13 }
14
15 signed main(){
16     ios::sync_with_stdio(false);
17     cin.tie(NULL);
18     int t = 1;
19     //cin >> t;
20     while(t-->0) solve();
21     return 0;
22 }

```

4 String

4.1 Hash

```

1 // String Hash template
2 // constructor(s) - O(|s|)
3 // query(l, r) - returns the hash of the range [l,r]
4 // from left to right - O(1)
5 // query_inv(l, r) from right to left - O(1)
6 // patrocinado por tiagodfs
7
8 mt19937 rng(time(nullptr));
9
10 struct Hash {
11     const int X = rng();
12     const int MOD = 1e9+7;
13     int n; string s;
14     vector<int> h, hi, p;
15     Hash() {}
16     Hash(string s): s(s), n(s.size()), h(n), hi(n), p
17     (n) {
18         for (int i=0;i<n;i++) p[i] = (i ? X*p[i-1]:1)
19         % MOD;
20         for (int i=0;i<n;i++)
21             h[i] = (s[i] + (i ? h[i-1]:0) * X) % MOD;
22         for (int i=n-1;i>=0;i--)
23             hi[i] = (s[i] + (i+1<n ? hi[i+1]:0) * X)
24             % MOD;
25     }
26 }

```

```

22 int query(int l, int r) {
23     int hash = (h[r] - (l ? h[l-1]*p[r-l+1]%MOD :
24         0));
25     return hash < 0 ? hash + MOD : hash;
26 }
27 int query_inv(int l, int r) {
28     int hash = (hi[l] - (r+1 < n ? hi[r+1]*p[r-l
29         +1] % MOD : 0));
30     return hash < 0 ? hash + MOD : hash;
31 }
32 };

```

5 DP

5.1 Lis

```

1 // Find the length of the longest increasing
2 // subsequence in a given array
3 // Time complexity: O(n log n) where n is the length
4 // of the input array
5
6 int lis(vector<int>& arr) {
7     vector<int> dp;
8     for (int x : arr) {
9         auto it = lower_bound(all(dp), x);
10        if(it == dp.end()) dp.push_back(x);
11        else *it = x;
12    }
13    return dp.size();
14 }

```

5.2 Countingdp

```

1 // Counting DP (Sum of all ways / Unbounded Knapsack)
2 // Time Complexity: O(N * Target)
3 // Space Complexity: O(Target)
4 // Use case: Count number of ways to form a sum
5
6 int count_ways(vector<int>& items, int target) {
7
8     vector<int> dp(target + 1, 0);
9     dp[0] = 1;
10
11     for (int item : items) {
12         for (int i = item; i <= target; i++) {
13             dp[i] = (dp[i] + dp[i - item]) % MOD;
14         }
15     }
16
17     return dp[target];
18 }

```

5.3 Knapsack

```

1 // General take/not take dp problem
2 // Time Complexity: aprox. O(n * m)
3 // Space Complexity: O(n * m) / O(m)
4 // Use cases: maximize/minimize sum
5
6 // Top-down
7 int knapsack_top_down(vector<int>& v, vector<int>& w,
8     int x){
9     int n = (int)v.size();
10    vector<vector<int>>> dp(n, vector<int>(x+1, -1));
11    auto rec = [&](auto self, int i, int m) -> int {
12        if(i >= n || m == 0) return 0;
13        if(dp[i][m] != -1) return dp[i][m];
14        int take = 0, not_take = self(self, i+1, m);
15        if(m >= w[i]){
16            take = v[i] + self(self, i+1, m - w[i]);
17        }
18    };
19 }

```

```

17     return dp[i][m] = max(take, not_take);
18 };
19
20 return rec(rec, 0, x);
21 }
22
23 // Bottom-up
24 int knapsack_bottom_up(vector<int>& v, vector<int>& w,
25     int x){
26     int n = (int)v.size();
27     vector<int> dp(x+1, 0);
28
29     for(int i=0; i < n; i++){
30         for(int m=x; m >= w[i]; m--){
31             dp[m] = max(dp[m], v[i] + dp[m - w[i]]);
32         }
33     }
34     return dp[x];
35 }

```

6 Math

6.1 Extendedeuclidean

```

1 // Extended Euclidean Algorithm
2 // Returns (gcd(a, b), x, y) such that a*x + b*y =
   gcd(a, b)
3
4 int euclides(int a, int b, int &x, int &y){
5     if(b == 0){
6         x = 1, y = 0;
7         return a;
8     }
9     int x1, y1;
10    int g = euclides(b, a%b, x1, y1);
11    x = y1; y = x1 - y1*(a/b);
12    return g;
13 }
14
15 int inverso_mod(int a, int m){
16     int x, y;
17     int g = euclides(a, m, x, y);
18     if(g != 1) return -1;
19     return (x%m + m) % m;
20 }

```

6.2 Crivo

```

1
2 // Time complexity: O(n log log n)
3 vector<int> spf(limit+1, 0);
4 void findPrimes(){
5     iota(all(spf), 0);
6     for(int i = 2; i*i <= limit; i++){
7         if(spf[i] == i){
8             for(int j=i*i; j <= limit; j += i){
9                 if(spf[j] == j) spf[j] = i;
10            }
11        }
12    }
13    // prime: spf[i] == i
14 }
15
16 // Time complexity: O(log n)
17 vector<int> getFactors(int x){
18     vector<int> factors;
19     while(x > 1){
20         factors.push_back(spf[x]);
21         x /= spf[x];
22     }

```

```

23     return factors;
24 }
25
26 // Time complexity: O(n log n)
27 vi tot_div(limit+1, 0);
28 void count_mult(){
29     for(int i=1; i <= limit; i++){
30         for(int j=i; j <= limit; j += i){
31             tot_div[j]++;
32         }
33     }
34 }

```

6.3 Fastexponentiation

```

1 // Calculates (x^n) % MOD
2 // Time Complexity: O(log n)
3 // Space Complexity: O(1)
4
5 int fexp(int x, int b, int MOD){
6     int res = 1;
7
8     while(b > 0){
9         if(b & 1) res = (res * x) % MOD;
10        x = (x * x) % MOD;
11        b >>= 1;
12    }
13    return res;
14 }

```

6.4 Matrixexponentiation

```

1 // Matrix Multiplication & Exponentiation Template
2 // Time Complexity: O(N^3) for multiplication, O(N^3
   * log P) for exponentiation
3 // Space Complexity: O(N^2)
4 // Use cases: Nth Fibonacci, paths of length K in a
   graph, linear recurrences
5
6 struct Matrix {
7     int r, c;
8     vector<vector<long long>> mat;
9
10    Matrix(int r, int c) : r(r), c(c) {
11        mat.assign(r, vector<long long>(c, 0));
12    }
13
14    Matrix operator*(const Matrix &other) const {
15
16        assert(c == other.r);
17
18        Matrix res(r, other.c);
19        for (int i = 0; i < r; i++) {
20            for (int k = 0; k < c; k++) {
21                if (mat[i][k] == 0) continue;
22                for (int j = 0; j < other.c; j++) {
23                    res.mat[i][j] = (res.mat[i][j] +
24                        mat[i][k] * other.mat[k][j]) % MOD;
25                }
26            }
27            return res;
28        }
29    };
30
31    Matrix power(Matrix base, long long exp) {
32        assert(base.r == base.c);
33        int n = base.r;
34        Matrix res(n, n);
35
36        for (int i = 0; i < n; i++) {
37            res.mat[i][i] = 1;

```

```

38     }
39
40     while (exp > 0) {
41         if (exp % 2 == 1) res = res * base;
42         base = base * base;
43         exp /= 2;
44     }
45     return res;
46 }

```

6.5 Pie

```

1 // inclusion exclusion principle using bitmaks
2 int count_union(vi &tot, vi &conj){
3     int m = (int)conj.size(), ans = 0;
4     int mx = (1ll << m) - 1;
5     for(int i=mx; i > 0; i--){
6         int cnt = __builtin_popcount(i), prod = 1;
7         for(int j=0; j<m; j++){
8             if(1 & (i >> j)) prod *= conj[j];
9         }
10
11         if(prod >= (int)tot.size()) continue;
12         if(cnt%2 == 0) ans -= tot[prod];
13         else ans += tot[prod];
14     }
15     return ans;
16 }

```

6.6 Factorial

```

1 // Time Complexity: O(N) for precompute, O(1) for
  choose and perm
2
3 const int MAXN, MOD;
4 vector<int> fact(MAXN), ifact(MAXN);
5
6 void precompute(){
7     fact[0] = 1; ifact[0] = 1;
8     for(int i=1; i<MAXN; i++) fact[i] = (i * fact[i
9 -1]) % MOD;
10    ifact[MAXN-1] = fexp(fact[MAXN-1], MOD-2, MOD);
11    for(int i=MAXN-2; i >= 0; i--) ifact[i] = (ifact[
12 i+1]*(i+1)) % MOD;
13 }
14
15 int choose(int n, int r){
16     if(r < 0 or r > n) return 0;
17     int num = fact[n];
18     int den = (ifact[r] * ifact[n - r]) % MOD;
19     return (num * den) % MOD;
20 }
21
22 int perm(int n, int r){
23     if(r < 0 or r > n) return 0;
24     return (fact[n] * ifact[n - r]) % MOD;
25 }

```

7 DS

7.1 Bit2d

```

1 // Time complexity: query and update log N * log M
2 // 1-based indexing
3
4 struct BIT2D {
5
6     int n, m; vector<vector<int>> bit;
7     BIT2D(int n, int m) {
8         this->n = n;
9         this->m = m;

```

```

10         bit.assign(n+1, vector<int>(m+1, 0));
11     }
12
13     void update(int x, int y, int val) {
14         for (int i = x; i <= n; i += i & -i) {
15             for (int j = y; j <= m; j += j & -j) {
16                 bit[i][j] += val;
17             }
18         }
19     }
20
21     int query(int x, int y) {
22         int sum = 0;
23         for (int i = x; i > 0; i -= i & -i) {
24             for (int j = y; j > 0; j -= j & -j) {
25                 sum += bit[i][j];
26             }
27         }
28         return sum;
29     }
30
31     int queryRange(int x1, int y1, int x2, int y2) {
32         return query(x2, y2)
33             - query(x1 - 1, y2)
34             - query(x2, y1 - 1)
35             + query(x1 - 1, y1 - 1);
36     }
37
38 };

```

7.2 Segtree

```

1 // 0-based indexing, [l, r)
2 // SegTree<int> seg;
3
4 template <typename T>
5 struct SegTree{
6
7     int size;
8     vector<T> tree;
9     const T NEUTRO = 0; // adapt
10
11     T merge(T a, T b){
12         return a + b; // adapt
13     }
14
15     void init(int n){
16         size = 1;
17         while(size < n) size *= 2;
18         tree.assign(2*size, NEUTRO);
19     }
20
21     void build(vector<T> &a, int x, int lx, int rx){
22         if(rx - lx == 1){
23             if(lx < (int)a.size()) tree[x] = a[lx];
24             return;
25         }
26         int m = (lx + rx)/2;
27         build(a, 2*x+1, lx, m);
28         build(a, 2*x+2, m, rx);
29         tree[x] = merge(tree[2*x+1], tree[2*x + 2]);
30     }
31
32     void build(vector<T> &a){
33         init((int)a.size());
34         build(a, 0, 0, size);
35     }
36
37     void set(int i, T v, int x, int lx, int rx){
38         if(rx - lx == 1){
39             tree[x] = v;
40             return;
41         }

```



```

42     int m = (lx + rx)/2;
43     if(i < m) set(i, v, 2*x+1, lx, m);
44     else set(i, v, 2*x + 2, m, rx);
45     tree[x] = merge(tree[2*x+1], tree[2*x+2]);
46 }
47
48 void set(int i, T v){
49     set(i, v, 0, 0, size);
50 }
51
52 T query(int l, int r, int x, int lx, int rx){
53     if(lx >= r or rx <= l) return NEUTRO;
54     if(lx >= l and rx <= r) return tree[x];
55     int m = (lx + rx)/2;
56     return merge(query(l, r, 2*x+1, lx, m), query
57 (l, r, 2*x+2, m, rx));
58 }
59
60 T query(int l, int r){
61     return query(l, r, 0, 0, size);
62 }
63 };

```

7.3 Seglazy

```

1 // 0-based indexing, [l, r)
2 // SegLazy<int> seg
3
4 template <typename T>
5 struct SegLazy {
6     int size;
7     vector<T> tree, lazy;
8     const T NEUTRO = 0, LAZY_NEUTRO = 0;
9
10    T merge(T a, T b) { return a + b; }
11
12    void init(int n) {
13        size = 1;
14        while (size < n) size *= 2;
15        tree.assign(2 * size, NEUTRO);
16        lazy.assign(2 * size, LAZY_NEUTRO);
17    }
18
19    void push(int x, int lx, int rx) {
20        if (lazy[x] == LAZY_NEUTRO) return;
21
22        tree[x] += lazy[x] * (rx - lx);
23
24        if (rx - lx > 1) {
25            lazy[2 * x + 1] += lazy[x];
26            lazy[2 * x + 2] += lazy[x];
27        }
28        lazy[x] = LAZY_NEUTRO;
29    }
30
31    void build(vector<T> &a, int x, int lx, int rx) {
32        if (rx - lx == 1) {
33            if (lx < (int)a.size()) tree[x] = a[lx];
34            return;
35        }
36        int m = (lx + rx) / 2;
37        build(a, 2 * x + 1, lx, m);
38        build(a, 2 * x + 2, m, rx);
39        tree[x] = merge(tree[2 * x + 1], tree[2 * x +
40 2]);
41
42    void build(vector<T> &a) {
43        init((int)a.size());
44        build(a, 0, 0, size);
45    }
46

```

```

47    void update(int l, int r, T val, int x, int lx,
48    int rx) {
49        push(x, lx, rx);
50        if (lx >= r or rx <= l) return;
51        if (lx >= l and rx <= r) {
52            lazy[x] += val;
53            push(x, lx, rx);
54            return;
55        }
56        int m = (lx + rx) / 2;
57        update(l, r, val, 2 * x + 1, lx, m);
58        update(l, r, val, 2 * x + 2, m, rx);
59        tree[x] = merge(tree[2 * x + 1], tree[2 * x +
60 2]);
61
62    void update(int l, int r, T val) {
63        update(l, r, val, 0, 0, size);
64    }
65
66    T query(int l, int r, int x, int lx, int rx) {
67        push(x, lx, rx);
68        if (lx >= r or rx <= l) return NEUTRO;
69        if (lx >= l and rx <= r) return tree[x];
70        int m = (lx + rx) / 2;
71        return merge(query(l, r, 2 * x + 1, lx, m),
72        query(l, r, 2 * x + 2, m, rx));
73    }
74
75    T query(int l, int r) {
76        return query(l, r, 0, 0, size);
77    }
78 };

```

7.4 Psum2d

```

1 // Psum2D
2 // Time Complexity: Query O(1)
3 // Space Complexity: O(N*M)
4 // Use cases: static 2d range sum, fixed-size window
5 // search, max sum submatrix
6
7 int n, m;
8 vector<vector<int>> arr(n+1, vector<int>(m+1, 0));
9 vector<vector<int>> psum(n+1, vector<int>(m+1, 0));
10
11 for (int i = 1; i <= n; i++) {
12     for (int j = 1; j <= m; j++) {
13         cin >> arr[i][j];
14         psum[i][j] = arr[i][j]
15             + psum[i][j-1]
16             + psum[i-1][j]
17             - psum[i-1][j-1];
18     }
19 }
20
21 // canto superior esquerdo (x1, y1) canto inferior
22 // direito (x2, y2)
23 int x1, y1, x2, y2;
24 int res = psum[x2][y2]
25     - psum[x1 - 1][y2]
26     - psum[x2][y1 - 1]
27     + psum[x1 - 1][y1 - 1];

```

7.5 Dsu

```

1 // Disjoint Set Union(DSU)
2 // Time Complexity: aprox. O(1)
3 // Space Complexity: O(n)
4 // Use cases: connected components, cycle detection,
5 // offline edge deletion

```

```

6 struct DSU{
7
8     vector<int> parent, size;
9
10    DSU(int n){
11        parent.assign(n+1, 0);
12        size.assign(n+1, 1);
13        iota(parent.begin(), parent.end(), 0);
14    }
15
16    int find(int v){
17        if(v == parent[v]) return v;
18        return parent[v] = find(parent[v]);
19    }
20
21    bool same(int a, int b){
22        return find(a) == find(b);
23    }
24
25    bool merge(int a, int b){
26        a = find(a);
27        b = find(b);
28        if(a != b){
29            if(size[a] < size[b]) swap(a, b);
30            parent[b] = a;
31            size[a] += size[b];
32            return true;
33        }
34        return false;
35    }
36 }
37 };

```

7.6 Sparsetable

```

1 // Sparse Table
2 // Time Complexity: O(n * log n) build, O(1) queries
3 // Space complexity: O(n * log n)
4 // Idempotent operation f(a, a) = a (min, max, gcd,
5 // and, or)
6 // Use cases: idempotent functions range queries
7 // 0-based indexing, [l, r]
8 struct SparseTable{
9
10    int n, p;
11    vector<vector<int>> st;
12
13    SparseTable(vector<int> &arr){
14        n = (int) arr.size();
15        p = __lg(n);
16        st.assign(p+1, vector<int>(n, 0));
17
18        for(int j=0; j<n; j++) st[0][j] = arr[j];
19        for(int i = 1; i <= p; i++){
20            int k = (1ll << (i-1));
21            int lim = n - 2*k + 1;
22            for(int j=0; j<lim; j++){
23                st[i][j] = op(st[i-1][j], st[i-1][j+k]);
24            }
25        }
26
27        int query(int l, int r){
28            int len = r - l + 1;
29            int p_local = __lg(len), k = (1ll << p_local);
30            return op(st[p_local][l], st[p_local][r - k + 1]);
31        }
32    }
33 };

```

7.7 Bit

```

1 // Time complexity: query and update log N
2 // 1-based indexing, [l, r]
3
4 struct BIT{
5
6     int n; vector<int> bit;
7     BIT(int n){
8         this->n = n;
9         bit.assign(n+1, 0);
10    }
11
12    void update(int i, int x){
13        while(i <= n){
14            bit[i] += x;
15            i += i & -i;
16        }
17    }
18
19    int query(int i){
20        int sum = 0;
21        while(i > 0){
22            sum += bit[i];
23            i -= i & -i;
24        }
25        return sum;
26    }
27
28    int queryRange(int l, int r){
29        return query(r) - query(l-1);
30    }
31
32    // retorna o indice do k-esimo elemento da bit
33    int find_kth(int k){
34
35        int idx = 0;
36        int max_bit = __lg(n);
37
38        for(int i = max_bit; i>= 0; i--){
39            int next_idx = idx + (1ll << i);
40            if(next_idx <= n and bit[next_idx] < k){
41                idx = next_idx;
42                k -= bit[next_idx];
43            }
44        }
45
46        return idx+1;
47    }
48 };
49 };

```

7.8 Monotonicstack

```

1 // Monotonic Stack, next greater number in a circular
2 // array
3 // Time Complexity: O(N)
4 // Space Complexity: O(N)
5 // Use cases: ring problems, finding next larger
6 // obstacle in a loop
7
8 for (int i = 0; i < 2 * n; i++) {
9     int num = a[i % n];
10    while (!st.empty() and a[st.top()] < num) { //
11        remove os menores
12        res[st.top()] = num; st.pop();
13    }
14
15    if (i < n) st.push(i);
16 }
17
18 // Monotonic Stack, next smaller element in a linear
19 // array

```

```
16 // Use cases: largest rectangle in histogram,
    subarray contribution
17
18 for (int i = 0; i < n; i++) {
19     while (!st.empty() and a[st.top()] > a[i]) { //
        remove os maiores
20         res[st.top()] = a[i];
21         st.pop();
22     }
23     st.push(i);
24 }
```